Valorisation des swaps OIS €STR

Théorie du pricing et implémentation dans le pricer de dérivés de taux

Projet `rate-derivatives-pricer` – phase 1

27 octobre 2026

Résumé

Ce document présente, d'une part, la théorie de la valorisation d'un swap de taux indexé sur un taux overnight (OIS) en monnaie euro : capitalisation du taux €STR, valorisation par actualisation sur une courbe unique, taux de par, annuité, construction de la courbe par *bootstrap*. Il expose, d'autre part, les choix de conception et l'implémentation de la phase 1 du projet : bibliothèque quantitative en Python et NumPy, service FastAPI, client Python typé et tableau de bord Streamlit. Chaque résultat chiffré (courbe, exemples, écarts) est produit par le code du dépôt, et l'implémentation est validée contre QuantLib avec des écarts de l'ordre de $2{,}4 \times 10^{-13}$ sur les facteurs d'actualisation.

Avertissement. Les données de marché utilisées (snapshot `EUR_2026-10-19`) sont *synthétiques* : aucun niveau de taux de ce document ne doit être lu comme une cotation réelle.

Table des matières

I Théorie : la valorisation d'un swap OIS **4**

1 Introduction **4**

2 Cadre et notations **4**
2.1 Facteurs d'actualisation, taux zéro et taux forward 4
2.2 Conventions de décompte 4
2.3 Calendriers et jours ouvrés 5

3 L'indice €STR et sa capitalisation **5**

4 Le swap OIS **5**
4.1 Structure 5
4.2 Conventions EUR retenues 6

5 Valorisation d'un swap OIS **6**
5.1 Hypothèse de courbe unique 6
5.2 Télescopage du coupon overnight 6
5.3 Valeur actuelle des deux jambes 7
5.4 Taux de par, annuité et PVBP 7
5.5 Exemple chiffré : un swap payeur à 5 ans 7
5.6 Swap en cours de vie 8

6 Construction de la courbe par bootstrap **8**
6.1 Principe 8
6.2 Interpolation 9
6.3 Résolution numérique 9

7 Dates : spot, maturité, échéancier **9**

II Implémentation et choix techniques **10**

8 Vue d'ensemble et principes **10**
8.1 Architecture en couches 10
8.2 Principes transversaux 11
8.3 Pipeline de calcul 11

9 Le cœur quantitatif **11**
9.1 Conventions : calendriers, décomptes, tenors 12
9.2 Échéanciers 13
9.3 Données de marché 13
9.4 Courbes et interpolation 13
9.5 Solveurs 13
9.6 Instruments de calibration et bootstrap 13
9.7 Instruments, jambes et coupons 14
9.8 Moteur de valorisation 15
9.9 Registres et erreurs 16

10 Services, API, SDK et interface **16**
10.1 Couche services 16

10.2 API HTTP 17
10.3 Journalisation 18
10.4 SDK et tableau de bord 18

11 Validation et qualité **19**
11.1 QuantLib comme oracle 19
11.2 Tolérances et résultats 19
11.3 Pyramide de tests 20
11.4 Chaîne de qualité 20
11.5 Incidents rencontrés 20

12 Journal des décisions **21**

13 Limites et suite **21**

A Calibration détaillée de la courbe **22**

B Carte des modules **22**

C Reproduire les chiffres et compiler **23**

Première partie

Théorie : la valorisation d'un swap OIS

1 Introduction

Un *swap OIS* (Overnight Indexed Swap) échange un taux fixe contre le taux overnight capitalisé jour après jour sur la période. Depuis la crise de 2008, c'est l'instrument de référence pour construire la courbe d'actualisation des produits collatéralisés : le taux overnight est le taux de rémunération du collatéral, de sorte que la courbe OIS est le taux « sans risque » du marché interbancaire [7, 5]. En zone euro, l'indice est l'€STR (*euro short-term rate*), publié par la BCE depuis octobre 2019 à partir des opérations de dépôt non garanties des banques ; il a remplacé l'EONIA, calculé d'octobre 2019 à janvier 2022 comme l'€STR majoré de 8,5 points de base.

Plan. La partie I est autonome : elle pose les notations (section 2), la capitalisation de l'€STR (section 3), le swap (section 4), sa valorisation (section 5), la construction de la courbe (section 6) et le calcul des dates (section 7). La partie II décrit comment ces objets sont codés, pourquoi les choix ont été faits ainsi, et comment le résultat est validé.

Périmètre. La phase 1 couvre le swap OIS en euro, avec une courbe unique (actualisation et projection sur la même courbe €STR). Les swaps contre Euribor, le multi-courbes et le calcul de risque (DV01, key-rate, convexité) sont prévus en phase 2 (section 13).

2 Cadre et notations

2.1 Facteurs d'actualisation, taux zéro et taux forward

Soit t_0 la date de valorisation. Le *facteur d'actualisation* (*discount factor*) $P(t)$ est la valeur en t_0 d'un euro reçu en $t \geq t_0$, avec $P(t_0) = 1$. La courbe est décrite par ses valeurs en des dates *piliers* et par un schéma d'interpolation. Le temps y est mesuré avec la convention Act/365F : $\theta(d) = (d - t_0)/365$ en années.

Définition 1 (Taux zéro, forward simple, forward instantané)**.** Pour une date $d > t_0$ et deux dates $s < e$ de longueur $\tau(s, e)$ selon une convention de décompte :

$$z(d) = -\frac{\ln P(d)}{\theta(d)} \quad \text{(taux zéro, capitalisation continue)}, \tag{1}$$

$$F(s, e) = \frac{1}{\tau(s, e)} \left(\frac{P(s)}{P(e)} - 1 \right) \quad \text{(forward simple)}, \tag{2}$$

$$f(d) = -\frac{\partial \ln P(d)}{\partial \theta} \quad \text{(forward instantané)}. \tag{3}$$

Le forward simple $F(s, e)$ est le taux qui, appliqué à l'euro placé en s jusqu'en e sans risque, rend l'opération équivalente à l'actualisation par la courbe : $1 + F(s, e)\,\tau(s, e) = P(s)/P(e)$.

2.2 Conventions de décompte

La fraction d'année $\tau(d_1, d_2)$ dépend d'une convention (définitions ISDA [6]). Celles retenues sont :

Convention	Définition de $\tau(d_1, d_2)$
Act/360	$(d_2 - d_1)/360$ (nombre exact de jours). Taux monétaires, €STR, jambes d'un OIS EUR.
Act/365F	$(d_2 - d_1)/365$. Mesure du temps des courbes (interpolation).
30/360 (Bond Basis)	$\dfrac{360(Y_2 - Y_1) + 30(M_2 - M_1) + (D'_2 - D'_1)}{360}$ avec $D'_1 = \min(D_1, 30)$ et $D'_2 = 30$ si $D_2 = 31$ et $D'_1 = 30$, sinon $D'_2 = D_2$.
30E/360 (Eurobond)	Même formule avec $D'_i = \min(D_i, 30)$.

2.3 Calendriers et jours ouvrés

Le calendrier TARGET compte comme jours fériés, en plus des week-ends : le 1er janvier, le vendredi saint, le lundi de Pâques, le 1er mai, le 25 et le 26 décembre (les trois derniers depuis 2000 ; le 31 décembre était aussi fermé en 1998, 1999 et 2001). Le dimanche de Pâques est calculé par l'algorithme de Meeus–Jones–Butcher ; le vendredi saint est ce dimanche moins deux jours et le lundi de Pâques plus un jour.

Une date tombant un jour non ouvré est déplacée selon une *convention de jour ouvré* : *Following* (jour ouvré suivant), *Preceding* (précédent), ou *Modified Following* : jour ouvré suivant, sauf s'il change de mois, auquel cas on recule au jour ouvré précédent. On note $\mathrm{adj}_c(d)$ la date ajustée selon la convention c et $\mathrm{adv}(d, n)$ la date située n jours ouvrés après d.

3 L'indice €STR et sa capitalisation

L'€STR est un taux d'intérêt d'un jour ouvré à l'autre. Le *fixing* du jour ouvré d est le taux r applicable de d au jour ouvré suivant ; il est publié le jour ouvré suivant, donc connu seulement a posteriori.

Sur une période de calcul $[S, E]$, soit $S = d_0 < d_1 < \cdots < d_n = E$ les jours ouvrés TARGET, r_i le fixing du jour d_i et $\delta_i = \tau_{360}(d_i, d_{i+1})$. Un investisseur qui replace chaque jour son capital au taux overnight obtient le facteur de capitalisation

$$C \;=\; \prod_{i=0}^{n-1} (1 + r_i\,\delta_i), \qquad R \;=\; \frac{C - 1}{\tau(S, E)}, \tag{4}$$

où R est le *taux overnight composé* de la période. Le montant d'intérêt sur un notionnel N est $N(C-1) = N\,R\,\tau(S,E)$. Cette capitalisation est dite *en arrears* : le taux n'est connu qu'à la fin de la période. Dans ce document, on n'applique ni *lookback*, ni *lockout*, ni *observation shift* : chaque fixing est utilisé le jour même.

4 Le swap OIS

4.1 Structure

Un swap OIS de notionnel N, de date de départ S et de maturité E échange deux jambes :

— la **jambe fixe**, qui verse à chaque date T_j^{fix} le coupon $N\,K\,\tau_j^{\mathrm{fix}}$, où K est le taux fixe ;

— la **jambe overnight**, qui verse à chaque date T_j le coupon $N\,(C_j - 1)$, où C_j est le facteur (4) de la période $[S_j, E_j]$.

Il n'y a pas d'échange de notionnel. Le détenteur d'un swap *payeur* paie le taux fixe et reçoit la jambe overnight ; un swap *receveur* fait l'inverse. Dans tout le document, la valeur actuelle (PV) est celle du point de vue du détenteur, et un montant reçu est compté positivement.

Chaque jambe est découpée en périodes $[S_j, E_j]$ (*accrual*) ; le paiement a lieu à $T_j = E_j$ augmenté d'un décalage de paiement (*payment lag*) en jours ouvrés.

4.2 Conventions EUR retenues

Ces conventions ont été fixées avant l'implémentation (décision ADR-019) :

Élément	Convention
Calendrier	TARGET
Décalage de départ (*spot lag*)	2 jours ouvrés après la date de valorisation
Fréquence des deux jambes	Annuelle (une seule période si la maturité est d'un an ou moins)
Décompte des deux jambes	Act/360
Convention de jour ouvré	Modified Following (dates intermédiaires et maturité)
Règle de fin de mois	Appliquée si le départ est le dernier jour ouvré du mois
Génération de l'échéancier	*Backward* depuis la maturité (période courte, ou *stub*, en tête)
Décalage de paiement	1 jour ouvré TARGET après la fin de la période
Capitalisation overnight	Quotidienne, en arrears, sans lookback, lockout ni observation shift
Dépôt overnight (calibration)	Départ en t_0, fin au jour ouvré suivant, Act/360, taux simple

5 Valorisation d'un swap OIS

5.1 Hypothèse de courbe unique

Le cadre est *mono-courbe* : une seule courbe P sert à actualiser les flux et à projeter les fixings futurs. Cette hypothèse est cohérente pour un OIS : l'indice est le taux de la courbe d'actualisation elle-même, de sorte que la capitalisation (4) se réplique par un dépôt overnight reconduit chaque jour. On pose donc, pour tout fixing futur, $r_i = F(d_i, d_{i+1})$ au sens de (2) avec $\tau = \tau_{360}$.

5.2 Télescopage du coupon overnight

Proposition 1 (Facteur de capitalisation projeté)**.** *Si tous les fixings de* $[S, E]$ *sont projetés sur la courbe, alors*

$$C \;=\; \prod_{i=0}^{n-1} \left(1 + F(d_i, d_{i+1})\, \delta_i\right) \;=\; \frac{P(S)}{P(E)}.$$

Démonstration. Par définition (2), $1 + F(d_i, d_{i+1})\, \delta_i = P(d_i)/P(d_{i+1})$. Le produit est télescopique : $\prod_{i=0}^{n-1} P(d_i)/P(d_{i+1}) = P(d_0)/P(d_n)$. □

La proposition 1 est une identité exacte et non une approximation : elle dispense de boucler sur les jours de la période (environ 250 par coupon annuel) et ramène le calcul d'un coupon à deux évaluations de la courbe. Une vérification numérique est donnée à la section 5.5.

Remarque. Si le paiement avait lieu en E sans décalage, la valeur de la jambe overnight serait $N(P(S)/P(E)-1)P(E) = N(P(S)-P(E))$: on retrouve la valeur d'un *floating rate note*. Le décalage de paiement $T_j > E_j$ rompt ce télescopage exact (voir ci-dessous) et le montant, connu en E_j, est simplement actualisé de T_j.

5.3 Valeur actuelle des deux jambes

Proposition 2 (PV des jambes)**.** *Soit m coupons de dates d'accrual $[S_j, E_j]$ et de paiement T_j. Les valeurs actuelles, non signées, des deux jambes sont*

$$V_{\text{fix}} = N\,K \sum_j \tau_j^{\text{fix}}\, P(T_j^{\text{fix}}) \;=\; K\,A, \tag{5}$$

$$V_{\text{ON}} = N \sum_{j=1}^{m} \left(\frac{P(S_j)}{P(E_j)} - 1 \right) P(T_j), \tag{6}$$

*où $A = N\sum_j \tau_j^{\text{fix}}\, P(T_j^{\text{fix}})$ est l'*annuité*, exprimée en unités de monnaie (le notionnel est inclus).*

Lorsque $T_j = E_j$ et que les périodes sont contiguës ($S_{j+1} = E_j$), la somme (6) se télescope en $N(P(S_1) - P(E_m))$. Avec un décalage de paiement, chaque terme est multiplié par le facteur $P(T_j)/P(E_j)$, proche de 1 ; le résultat n'a plus de forme fermée aussi simple mais reste calculé exactement.

5.4 Taux de par, annuité et PVBP

Définition 2 (Taux de par)**.** Le *taux de par* (ou taux de swap) est le taux fixe S_0 pour lequel le swap a une valeur nulle : $V_{\text{fix}} = V_{\text{ON}}$, soit

$$S_0 \;=\; \frac{V_{\text{ON}}}{A}. \tag{7}$$

La valeur d'un swap payeur de taux fixe K est alors

$$V_{\text{payeur}} \;=\; V_{\text{ON}} - V_{\text{fix}} \;=\; A\,(S_0 - K), \qquad V_{\text{receveur}} = -V_{\text{payeur}}. \tag{8}$$

Le taux de par est ce que le marché cote pour chaque maturité ; la formule (8) montre que la valeur du swap est l'écart au par, multiplié par l'annuité. Le *PVBP* (*present value of a basis point*) est la variation de la jambe fixe pour un point de base de taux fixe : $\text{PVBP} = A \times 10^{-4}$, de sorte que $\partial V_{\text{payeur}}/\partial K = -A$.

5.5 Exemple chiffré : un swap payeur à 5 ans

Prenons, à la date de valorisation 2026-10-19, un swap payeur de notionnel $N = 10\,000\,000$ €, de taux fixe $K = 2{,}25\,\%$ et de maturité 5 ans, sur la courbe €STR du snapshot synthétique. Le swap démarre au 2026-10-21 (spot) et arrive à échéance le 2031-10-21 ; il compte 10 flux futurs, soit 5 coupons par jambe. Le pricer renvoie :

Grandeur	Valeur
Taux de par S_0	2,2540 %
Annuité A	47 563 443,37 €
PVBP $= A \times 10^{-4}$	4 756,34 €
PV de la jambe fixe (payée)	−1 070 177,48 €
PV de la jambe overnight (reçue)	1 072 080,01 €
PV du swap payeur	**1 902,54 €**

La relation (8) se vérifie : $A\,(S_0 - K) = 1\,902{,}54\,€$, identique à la PV renvoyée.

Vérification du télescopage. Pour le premier coupon overnight de ce swap (du 2026-10-21 au 2027-10-21), le produit des 257 taux overnight implicites de la courbe, calculés un à un sur les jours ouvrés TARGET, vaut 1,01911180555556, et le rapport $P(S)/P(E)$ vaut 1,01911180555556 ; l'écart est de $1{,}3 \times 10^{-15}$, c'est-à-dire de l'ordre de l'erreur d'arrondi sur 257 multiplications en double précision.

5.6 Swap en cours de vie

Pour un swap déjà commencé à la date t_0, un coupon overnight $[S_j, E_j]$ peut être en partie réalisé. Soit d_k le premier jour d'observation dont le fixing n'est pas encore publié. Alors

$$C_j \;=\; \underbrace{\prod_{d_i < d_k} (1 + r_i\,\delta_i)}_{\text{fixings publiés}} \;\times\; \underbrace{\frac{P(d_k)}{P(E_j)}}_{\text{partie projetée}} \quad , \tag{9}$$

où la partie projetée utilise de nouveau la proposition 1. Les règles suivantes s'appliquent (décision ADR-020) :

— les fixings *strictement antérieurs* à t_0 sont obligatoires ; leur absence est une erreur de données de marché ;

— le fixing du jour t_0 est utilisé s'il est fourni, sinon il est projeté ;

— un flux payé en t_0 ou avant est échu et n'entre pas dans la PV.

6 Construction de la courbe par bootstrap

6.1 Principe

Le marché cote les taux de par de swaps OIS de maturités $1\mathrm{W}, 2\mathrm{W}, \ldots, 30\mathrm{Y}$ ainsi que le taux du dépôt overnight. La courbe P est déterminée par ses valeurs aux *piliers* $T_1 < \cdots < T_K$: le pilier d'un dépôt est sa maturité, celui d'un OIS est sa *dernière date de paiement*, qui est la date la plus lointaine dont dépend son taux de par.

Définition 3 (Équations de calibration)**.** Pour chaque instrument k de cotation q_k, on cherche $P_k = P(T_k)$ tel que la cotation implicite de la courbe égale la cotation de marché :

— dépôt overnight, de $s = t_0$ à e : $q = \dfrac{1}{\tau_{360}(s,e)}\Big(\dfrac{1}{P(e)} - 1\Big)$, d'où la solution fermée $P(e) = 1/(1 + q\,\tau_{360}(s,e))$;

— OIS de maturité k : $S_0(P_1, \ldots, P_k) = q_k$, avec S_0 donné par (7).

L'équation de l'OIS n'a pas de solution fermée : les facteurs d'actualisation aux dates intermédiaires sont interpolés entre piliers, et le taux de par dépend de P_k de manière non linéaire. On la résout donc numériquement, pilier par pilier, dans l'ordre croissant des maturités.

Pourquoi une seule passe suffit. Avec une interpolation *locale* (la valeur interpolée sur un segment $[T_{k-1}, T_k]$ ne dépend que des nœuds P_{k-1} et P_k), le taux de par de l'instrument k

ne dépend que des nœuds $P_1, \dots, P_k$, puisque tous ses flux sont datés au plus tard en T_k. Les nœuds déjà trouvés ne sont donc jamais remis en cause : l'algorithme est séquentiel et exact, sans itération globale.

6.2 Interpolation

Soit $\theta_k = \theta(T_k)$, avec $\theta_0 = 0$. Deux schémas sont disponibles :

— **Log-linéaire sur les facteurs d'actualisation** (par défaut) : pour $\theta \in [\theta_{k-1}, \theta_k]$,

$$\ln P = \ln P_{k-1} + \frac{\theta - \theta_{k-1}}{\theta_k - \theta_{k-1}} (\ln P_k - \ln P_{k-1}),$$

ce qui revient à un forward instantané f *constant par morceaux*. C'est le standard des courbes OIS [4].

— **Linéaire sur le taux zéro** : z est linéaire en θ entre les nœuds, avec $z(\theta = 0)$ pris égal au zéro du premier pilier.

Au-delà du dernier pilier θ_K, les deux schémas prolongent la courbe à *forward instantané constant*. Pour le schéma log-linéaire, c'est le forward du dernier segment ; pour le schéma linéaire sur le zéro, c'est $f_K = z_K + \theta_K\, z'(\theta_K)$, d'où $-\ln P = z_K\, \theta_K + f_K\, (\theta - \theta_K)$.

6.3 Résolution numérique

Pour le pilier k, on cherche la racine de $g(P) = S_0(P_1, \dots, P_{k-1}, P) - q_k$ dans l'intervalle

$$\left[P_{k-1}\, e^{-1{,}0\, \Delta\theta},\ P_{k-1}\, e^{+0{,}5\, \Delta\theta} \right], \qquad \Delta\theta = \theta_k - \theta_{k-1},$$

qui correspond à des forwards continus compris entre $-50\,\%$ et $+100\,\%$: large pour tout marché réaliste, mais suffisamment serré pour que g change de signe. Plus P est faible, plus les taux implicites sont élevés ; g est en pratique strictement décroissante, ce qui rend la racine unique dans l'intervalle.

Méthode de Brent. Brent [3] combine la bissection (robuste, convergence linéaire), la sécante et l'interpolation quadratique inverse (convergence superlinéaire). À chaque itération, le pas d'interpolation n'est accepté que s'il reste à l'intérieur de l'encadrement et réduit suffisamment l'intervalle ; sinon on bissecte. La convergence est garantie dès que $g(P_{\min})$ et $g(P_{\max})$ sont de signes opposés.

Méthode de Newton protégée. Une variante de Newton–Raphson, avec dérivée par différences finies centrées, est disponible en option ; tout pas sortant de l'encadrement est remplacé par une bissection, ce qui lui conserve la robustesse de la bissection.

Critère d'échec. Un pilier dont le résidu final $|S_0 - q_k|$ dépasse 10^{-12} (en taux décimal) est une erreur de calibration, qui identifie le pilier fautif et son résidu.

7 Dates : spot, maturité, échéancier

Date spot. Avec le calendrier $\mathcal{C}$ et le décalage $\ell = 2$ jours ouvrés : $t_{\text{spot}} = \text{adv}(\text{adj}_F(t_0),\, \ell)$, où F désigne *Following*.

Maturité. Si la règle de fin de mois s'applique (le spot est le dernier jour ouvré de son mois), la maturité d'un swap de tenor n mois est le dernier jour ouvré du mois situé n mois plus tard. Sinon, c'est $t_{\text{spot}} + n$ mois, ajustée ensuite par l'échéancier.

Échéancier (génération *backward*).

1. À partir de la date de fin, on recule par pas de la fréquence (1 an) tant que la date obtenue n'est pas antérieure à la date de départ ; le reliquat plus court que la fréquence est une période courte (*stub*), placée en tête.
2. Si la règle de fin de mois s'applique, les dates intermédiaires et la date de fin sont ramenées au dernier jour ouvré de leur mois ; la date de départ est seulement ajustée.
3. Sinon, chaque date est ajustée en Modified Following.
4. La date de paiement d'une période est la date de fin ajustée, avancée de 1 jour ouvré TARGET.

Illustration. Pour un swap payeur à 2 ans de notionnel 10 000 000 € et de taux fixe 2,00 %, le tableau 1 donne l'échéancier et les flux calculés. La deuxième période se termine le 2028-10-23 et non le 2028-10-21, qui est un samedi (Modified Following), et le paiement a lieu le jour ouvré suivant. Le taux de la jambe overnight est le taux composé (4) de chaque période ; le taux de par vaut 1,9920 %, l'annuité 19 765 217,95 € et la PV du swap payeur −1 581,22 €.

Table 1 − Flux d'un swap payeur à 2 ans (notionnel 10^7 €, $K = 2{,}00\,\%$). Montants signés : positifs si reçus.

Jambe	Début	Fin	Paiement	Taux (%)	Montant (€)	DF	PV (€)
Fixe (pay)	2026-10-21	2027-10-21	2027-10-22	2,0000	−202 777,78	0,981090	−198 943,29
Fixe (pay)	2027-10-21	2028-10-23	2028-10-24	2,0000	−204 444,44	0,960462	−196 361,07
Overnight (receive)	2026-10-21	2027-10-21	2027-10-22	1,8850	191 118,06	0,981090	187 504,05
Overnight (receive)	2027-10-21	2028-10-23	2028-10-24	2,1004	214 708,28	0,960462	206 219,09

Deuxième partie

Implémentation et choix techniques

8 Vue d'ensemble et principes

8.1 Architecture en couches

Le projet est un paquet Python unique, `rate_derivatives_pricer`, organisé en cinq couches dont les dépendances vont dans *un seul sens* (figure 1).

La règle est vérifiée en intégration continue par cinq contrats `import-linter` : (1) la hiérarchie ui → sdk → api → services → core ; (2) `ui` n'atteint jamais `services` ni `core`, même indirectement ; (3) `ui` n'importe jamais `api` directement ; (4) `sdk` ne descend pas sous l'API ; (5) `core` ne dépend d'aucun paquet web, d'interface, de SciPy ou de QuantLib. Un import interdit fait échouer la CI.

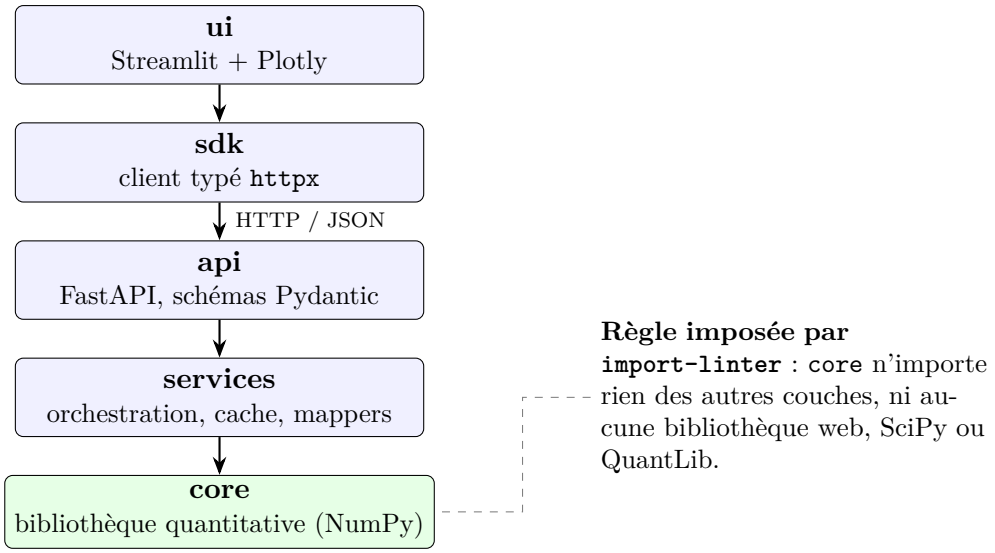


Figure 1 – Couches du projet. Les flèches indiquent le sens de la dépendance.

8.2 Principes transversaux

— **Immutabilité.** Instruments, cotations, échéanciers, snapshots et résultats sont des `dataclass` figées (`frozen=True`, `slots=True`). Un choc de risque produira de nouveaux objets plutôt que de modifier les anciens.

— **Pas d'état global.** La date de valorisation est une donnée explicite du `MarketSnapshot`. Le `Settings.evaluationDate` global de QuantLib n'est volontairement pas reproduit (ADR-009) ; les tests alignent QuantLib sur la date du snapshot.

— **Données et comportement séparés.** Un instrument décrit un contrat, un moteur le valorise, un schéma d'API le transporte. Les instruments ne portent pas leur moteur, contrairement à QuantLib (ADR-007).

— **Conventions en configuration.** Aucune convention de marché n'est codée en dur dans le calcul : elles sont lues dans un fichier JSON par devise (ADR-002, ADR-019).

— **Erreurs typées.** Une exception par mode d'échec, traduite en code HTTP stable.

— **Le cœur émet les logs, l'API les configure.** `core` n'utilise que le module `logging` standard.

8.3 Pipeline de calcul

Le flux est fonctionnel : le snapshot est transformé en environnement de marché par le bootstrap, puis l'instrument est valorisé dans cet environnement par un moteur sans état (figure 2). Le risque se ramènera à « modifier le snapshot, puis rejouer le pipeline ».

9 Le cœur quantitatif

Le cœur ne dépend que de NumPy et de la bibliothèque standard. Il compte les sous-paquets suivants (la liste complète des modules figure en annexe B).

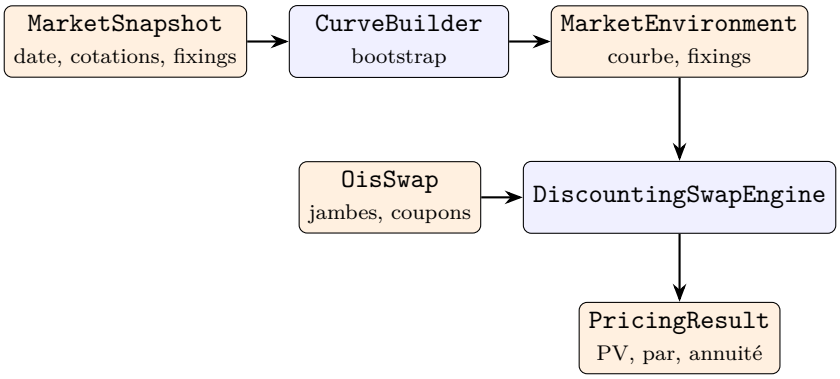


Figure 2 – Pipeline fonctionnel du cœur : chaque étape prend des entrées et renvoie une sortie.

9.1 Conventions : calendriers, décomptes, tenors

`core/conventions` implémente les objets de la section 2. `Calendar` est une classe abstraite dont dérivent `TargetCalendar`, `NullCalendar`, `WeekendsOnlyCalendar` et `JointCalendar` (union des jours fériés). Les jours fériés TARGET sont calculés par règles (Pâques par l'algorithme de Meeus) et mis en cache, car le calendrier est interrogé jour par jour. `Tenor` (« 3M », « 10Y », « ON ») et `Frequency` sont des objets valeur ; l'addition de mois ramène le jour à la longueur du mois cible (31 janvier + 1 mois = 28 ou 29 février).

Les classes `Act360`, `Act365Fixed`, `Thirty360` et `Thirty360European` se déclarent dans un registre par un décorateur :

```
@DAY_COUNTS.register("ACT/360")
@dataclass(frozen=True, slots=True)
class Act360(DayCountConvention):
    name: ClassVar[str] = "ACT/360"

    def day_count(self, start: date, end: date) -> int:
        return (end - start).days

    def year_fraction(self, start: date, end: date) -> float:
        return self.day_count(start, end) / DAYS_PER_YEAR_360
```

Listing 1 – Enregistrement d'un composant par décorateur (principe Open/Closed).

Ajouter un composant revient donc à ajouter un fichier, sans modifier de ligne existante. Deux variantes 30/360 explicites (Bond Basis et Eurobond) remplacent une classe `Thirty360` ambiguë (ADR-019).

Conventions de devise. `CurrencyConventions` regroupe calendrier, *spot lag*, mesure du temps des courbes, indice overnight, conventions du dépôt et de l'OIS. Elle est construite à partir d'un fichier JSON par devise (listing 2) ; l'ajout de l'USD en phase 3 se fera par un nouveau fichier, sans toucher au code de calcul.

```
{
  "currency": "EUR",
  "calendar": "TARGET",
  "spot_lag_days": 2,
  "curve_day_count": "ACT/365F",
  "overnight_index": { "name": "ESTR", "calendar": "TARGET", "day_count": "ACT/360" },
  "deposit": { "settlement_days": 0, "business_day_convention": "MODIFIED_FOLLOWING",
               "end_of_month": false, "day_count": "ACT/360" },
  "ois": { "fixed_frequency": "ANNUAL", "fixed_day_count": "ACT/360",
           "floating_frequency": "ANNUAL", "floating_day_count": "ACT/360",
           "business_day_convention": "MODIFIED_FOLLOWING", "end_of_month": true,
           "rule": "BACKWARD", "payment_lag_days": 1 }
}
```

Listing 2 – `core/conventions/data/EUR.json`.

9.2 Échéanciers

`ScheduleGenerator` transforme (départ, fin, fréquence) en une suite de `SchedulePeriod` (début et fin d'accrual, date de paiement, indicateur de période courte). Il suit pas à pas l'algorithme de la section 7. La comparaison date par date avec QuantLib a révélé un détail de convention : en génération *backward* avec la règle de fin de mois, QuantLib ne déplace *pas* la date de départ en fin de mois, seule la date « pivot » l'est. Notre première version déplaçait les deux ; le test de validation l'a détecté et le générateur a été corrigé.

9.3 Données de marché

— `Quote` (`DepositQuote`, `OisSwapQuote`) porte *toujours une unité* explicite (`percent`, `decimal` ou `bp`) ; la propriété `rate` convertit en décimal. Confondre 2,5 et 0,025 est un bug de production classique, que l'unité obligatoire rend impossible.

— `MarketSnapshot` regroupe la date de valorisation, les cotations et les fixings. Il refuse les cotations en double et les fixings postérieurs à la date de valorisation. C'est l'*unité de choc* du futur calcul de risque.

— `FileMarketDataProvider` lit `data/market/<nom>.json`. Les noms sont filtrés par expression régulière pour interdire toute remontée d'arborescence (`../`). Le format canonique du snapshot sert aussi de base à l'empreinte du cache.

9.4 Courbes et interpolation

`DiscountCurve` est définie par des nœuds (date, facteur d'actualisation), un interpolateur et une convention de décompte interne. Le temps de chaque nœud est converti une fois, à la construction, en tableaux NumPy mis en cache dans l'objet figé. Les interpolateurs (`LogLinearOnDiscountFactor`, `LinearOnZeroRate`) sont deux stratégies interchangeables derrière une interface `Interpolator` qui travaille sur des tableaux : l'évaluation d'une courbe sur une grille de dates est vectorisée.

9.5 Solveurs

`BrentSolver` et `NewtonSolver` implémentent l'interface `Solver` (pas de SciPy dans le cœur, ADR-004). Ils renvoient un `SolverResult` (racine, résidu, itérations, convergence) plutôt que de lever une exception : c'est le bootstrap qui décide de la gravité d'un échec. La tolérance absolue par défaut est de 10^{-15} sur le facteur d'actualisation, et le nombre maximal d'itérations de 100.

9.6 Instruments de calibration et bootstrap

L'abstraction `CalibrationInstrument` est l'équivalent du `RateHelper` de QuantLib : pour une courbe candidate, elle calcule la cotation implicite et indique le pilier qu'elle détermine. `DepositCalibration` renvoie le forward simple entre début et fin ; `OisCalibration` construit un swap de notionnel 1 aux conventions de la devise et renvoie son taux de par. Cette abstraction

rend le bootstrap générique : ajouter un instrument (un FRA, par exemple) n'exige aucune modification de `CurveBuilder`.

`CurveBuilder.build(snapshot)` trie les instruments par pilier, refuse deux instruments sur le même pilier, puis résout chaque nœud avec le solveur. Le cœur de la boucle est le suivant (extrait, journalisation omise) :

```
market_quote = instrument.quote.rate
previous_df = discount_factors[-1] if discount_factors else 1.0
dt = template.time(pillar) - previous_time

def objective(discount_factor: float) -> float:
    candidate = template.with_pillars(dates, (*discount_factors, discount_factor))
    return instrument.implied_quote(environment(candidate)) - market_quote

result = self._solver.solve(
    objective,
    lower=previous_df * math.exp(-BRACKET_MAX_FORWARD_RATE * dt),
    upper=previous_df * math.exp(-BRACKET_MIN_FORWARD_RATE * dt),
    initial_guess=previous_df * math.exp(-market_quote * dt),
)
if not result.converged or not abs(result.residual) <= CALIBRATION_RESIDUAL_TOLERANCE:
    raise CalibrationError(
        f"bootstrap failed on pillar {instrument.label}",
        pillar=instrument.label, residual=result.residual,
    )
```

Listing 3 – Calibration d'un pilier (`core/market/bootstrap.py`).

Le résultat est un `MarketEnvironment` : la courbe (qui sert à la fois d'actualisation et de projection de l'indice `ESTR`), les fixings et un `CalibrationReport` qui détaille chaque pilier (cotation, cotation implicite, résidu, itérations). Sur le snapshot de référence (24 cotations), la calibration compte 24 piliers, au plus 9 itérations par pilier et un résidu maximal de $6{,}0 \times 10^{-15}$, en environ 0,1 s. La figure 3 montre la courbe obtenue.

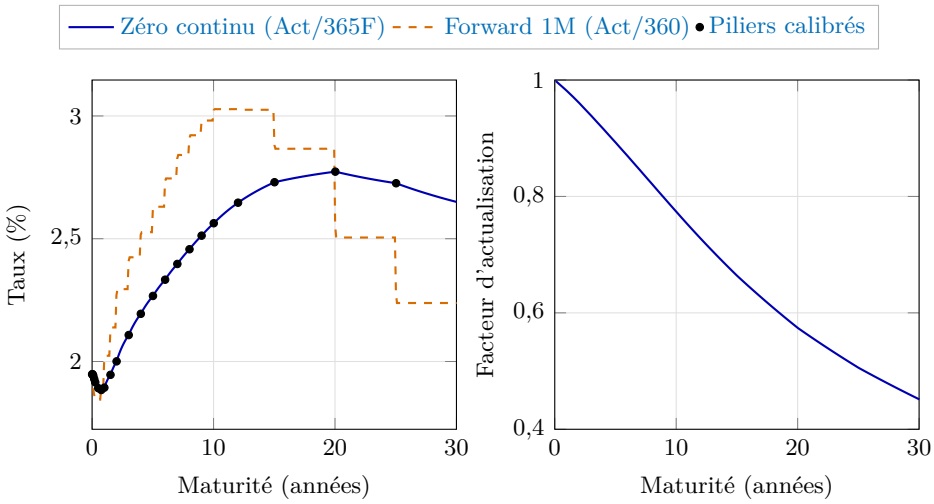


Figure 3 – Courbe €STR bootstrapée sur le snapshot synthétique du 2026-10-19, tracée à partir des données produites par le pricer.

9.7 Instruments, jambes et coupons

— `Cashflow` est une classe abstraite : chaque flux *calcule lui-même son montant* à partir du `MarketEnvironment` (ADR-007). `Coupon` (notionnel, dates d'accrual, date de paiement, décompte) en dérive, avec `FixedCoupon` et `OvernightCompoundedCoupon`.

— `Leg` regroupe des flux, un notionnel et un sens (`PayReceive`, signe ±1) : la composition

plutôt que l'héritage.

— `OisSwap` décrit le contrat ; `SwapDirection` (`pay_fixed` ou `receive_fixed`) est nommé d'après la jambe fixe.

— `factories.py` construit un swap à partir des conventions : `spot_date`, `ois_maturity_date` (qui reproduit la logique de `MakeOIS`, règle de fin de mois comprise), `build_ois_swap` (deux échéanciers puis deux jambes) et la factory `ois_swap_factory` enregistrée sous la clé `ois_swap`.

Le coupon overnight est la pièce la plus délicate. Il applique l'équation (9) : boucle jour par jour sur la partie réalisée, télescopage sur la partie projetée (proposition 1).

```python
def compound_factor(self, env: MarketEnvironment) -> float:
    valuation_date = env.valuation_date
    curve = env.projection_curve(self.index.name)
    if self.accrual_start > valuation_date:
        # Entirely in the future: telescoped projection.
        start_df, end_df = curve.discount_factors([self.accrual_start, self.accrual_end])
        return float(start_df / end_df)

    observation_dates = [d for d in self.index.fixing_calendar.business_days(
        self.accrual_start, self.accrual_end) if d < self.accrual_end]
    period_ends = [*observation_dates[1:], self.accrual_end]

    factor, i, n = 1.0, 0, len(observation_dates)
    # Realised part: fixings strictly before the valuation date are mandatory.
    while i < n and observation_dates[i] < valuation_date:
        factor *= self._realised_growth(env, observation_dates[i], period_ends[i])
        i += 1
    # Fixing of the valuation date itself: used if already known, projected otherwise.
    if i < n and observation_dates[i] == valuation_date:
        todays_fixing = env.fixing(self.index.name, valuation_date)
        if todays_fixing is not None:
            factor *= 1.0 + todays_fixing * self.index.day_count.year_fraction(
                observation_dates[i], period_ends[i])
            i += 1
    # Projected part, telescoped on the projection curve.
    if i < n:
        start_df, end_df = curve.discount_factors([observation_dates[i], self.accrual_end])
        factor *= float(start_df / end_df)
    return factor
```

Listing 4 – Facteur de capitalisation d'un coupon overnight (`cashflows.py`, docstrings omises).

Un fixing passé manquant lève une `MarketDataError` qui précise l'indice et la date, et se traduit côté API par une erreur 422 de code `MARKET_DATA_INVALID`.

9.8 Moteur de valorisation

`PricingEngine` est une interface ; `engine_for(type, method)` choisit l'implémentation dans le registre `ENGINES`. Les moteurs sont sans état. `DiscountingSwapEngine` implémente les formules (5)–(8) :

```python
def _alive_coupons(leg: Leg, env: MarketEnvironment) -> list[Coupon]:
    # Coupons of the leg paid strictly after the valuation date.
    return [cf for cf in leg.cashflows
            if isinstance(cf, Coupon) and cf.payment_date > env.valuation_date]

def par_rate(self, instrument: Instrument, env: MarketEnvironment) -> float:
    swap = self._check(instrument)
    annuity = _annuity(swap.fixed_leg, env)          # sum N * tau_i * P(pay_i)
    if annuity == 0.0:
        raise PricingError("par rate undefined: no remaining fixed coupon")
    return _unsigned_leg_value(swap.overnight_leg, env) / annuity
```

Listing 5 – Fonctions de base du moteur (`engines/discounting.py`).

La méthode `price` renvoie un `PricingResult`, *jamais un simple flottant* : PV total, PV par jambe, tableau des flux (dates, fraction d'année, taux, montant signé, facteur d'actualisation, PV), taux de par, annuité, PVBP et diagnostics (moteur, durée, version du cœur, avertissements). Si aucun coupon fixe ne subsiste, le taux de par est indéfini : il vaut `NaN` dans le cœur, un avertissement est émis, et l'API renvoie `null`. La méthode `par_rate` est la version allégée, sans tableau de flux, réutilisée par le bootstrap.

9.9 Registres et erreurs

Six registres (`CALENDARS`, `DAY_COUNTS`, `INTERPOLATORS`, `INSTRUMENTS`, `ENGINES`, `CONVENTIONS`) associent une clé stable à un composant. Leurs modules sont importés *paresseusement* à la première consultation : un module de calendrier importe le registre pour s'y enregistrer, donc le registre ne peut pas importer les calendriers au chargement sans créer d'import circulaire. Le chargeur est commun à tous les registres et réentrant. Le catalogue de l'API (`/v1/catalog`) est généré à partir de ces registres.

La hiérarchie d'erreurs `RatesPricerError` comporte une classe par mode d'échec (`ConventionError`, `ScheduleError`, `MarketDataError`, `CalibrationError`, `UnsupportedInstrumentError`, `PricingError`, `RiskComputationError`). Chacune porte un message et des détails structurés, que l'API met en correspondance avec un code HTTP (tableau 3).

10 Services, API, SDK et interface

10.1 Couche services

`PricerService` est la façade appelée par l'API. Ses entrées sont des dataclasses figées (`SnapshotReference` ou `InlineMarket` pour le marché, `TradeInput`, `CalculationOptions`) : Pydantic reste cantonné à `api` et `sdk` (ADR-021). Pour chaque calcul, le service :

1. charge le snapshot, par référence (fichier) ou par valeur (corps de la requête) ;
2. calcule l'*empreinte* du marché et consulte le cache de courbes ;
3. en cas d'absence, bootstrape la courbe et la met en cache ;
4. construit l'instrument avec la factory, le valorise avec le moteur ;
5. renvoie le résultat accompagné de diagnostics (durées par étape, *hit* ou *miss* du cache, avertissements, empreintes) et émet un événement d'audit.

Cache de courbes. `LruCurveCache` est un cache en mémoire, *least recently used*, à capacité configurable (32 par défaut) et à durée de vie optionnelle, protégé par un verrou (l'API sert plusieurs requêtes dans des threads). Il tient des compteurs (*hits*, *misses*, évictions) et accepte une horloge injectable, ce qui permet de tester l'expiration sans attente. Il est *par processus* : avec plusieurs *workers*, chacun a son cache. Une interface `CurveCache` permettra de le remplacer par Redis (ADR-005).

Clé du cache. La clé est le SHA-256 de la sérialisation JSON canonique (clés triées) des données du snapshot (date, devise, cotations avec leurs unités, fixings), de l'interpolation, du solveur et de la version du cœur. Le *nom* du snapshot en est exclu : les mêmes données passées

par référence ou par valeur partagent la même entrée (ADR-026). Une même empreinte d'entrée et une même version doivent donner exactement le même résultat ; un PV différent révèle alors un bug.

10.2 API HTTP

L'API FastAPI expose, en phase 1, les routes du tableau 2. Principes du contrat :

Table 2 – Routes de l'API en phase 1.

Route	Rôle
`GET /health`	Liveness : le processus répond.
`GET /ready`	Readiness : conventions et registres chargés (503 sinon).
`GET /v1/version`	Versions de l'API, du cœur, de Python et de NumPy ; commit déployé.
`GET /v1/catalog`	Types d'instruments, interpolations, devises, schémas des trades, générés depuis les registres.
`GET /v1/market-snapshots`	Liste des snapshots ; le contenu d'un snapshot est sous `/v1/market-snapshots/<nom>`.
`POST /v1/curves/build`	Bootstrap : nœuds calibrés, grille de la courbe, diagnostics de convergence.
`POST /v1/price`	Valorisation d'un trade.

— **Sans état** : chaque requête porte ou référence ses données de marché et son trade.

— **Versionnée par l'URL** (`/v1`) : un changement incompatible crée `/v2`.

— **DTO distincts du domaine** : les schémas Pydantic (`api/schemas.py`) ne dépendent que de Pydantic ; les champs inconnus sont refusés.

— **Unités explicites** : dates ISO 8601, taux en décimal, cotations avec unité.

— **Enveloppe permanente** : chaque réponse contient `request_id`, `result` et un bloc `diagnostics`.

```
POST /v1/price
{ "market":  { "snapshot": "EUR_2026-10-19" },
  "trade":   { "type": "ois_swap",
               "params": { "notional": 10000000, "fixed_rate": 0.0225,
                           "direction": "pay_fixed", "maturity": "5Y" } },
  "options": { "interpolation": "log_linear_df" } }

200 OK
{ "request_id": "smoke-1",
  "result": { "instrument_type": "ois_swap", "currency": "EUR",
              "present_value": 1902.5377349308692, "par_rate": 0.02254000000000001,
              "annuity": 47563443.373254724, "legs": [ ... ], "cashflows": [ ... ] },
  "diagnostics": { "curve_cache": "miss", "timings_ms": { "curve_build": 99.4, "pricing": 0.7 },
                   "versions": { "api": "0.1.0", "core": "0.1.0" }, "warnings": [],
                   "input_hashes": { "market": "8295394e...", "trade": "dc6b132c..." } } }
```

Listing 6 – Requête `POST /v1/price` et réponse (abrégée).

Gestion des erreurs. Un format unique est utilisé partout, `{"request_id", "error": {"code", "message", "details"}}`, avec un code machine stable (tableau 3). Une exception

inattendue produit une réponse générique `INTERNAL_ERROR` sans trace ; la trace complète est journalisée avec le `request_id`.

Table 3 – Correspondance entre erreurs du cœur et réponses HTTP.

Source	HTTP	Code
Validation Pydantic (JSON mal formé, champ manquant ou inconnu)	422	`VALIDATION_ERROR`
`MarketDataError`	422	`MARKET_DATA_INVALID`
`ConventionError`, `ScheduleError`	422	`INVALID_TRADE_DEFINITION`
`UnsupportedInstrumentError`	400	`UNSUPPORTED_INSTRUMENT`
`CalibrationError` (pilier et résidu)	422	`CALIBRATION_FAILED`
`MarketDataNotFoundError`, route inconnue	404	`NOT_FOUND`
`PricingError`, `RiskComputationError`	500	`COMPUTATION_FAILED`
Exception inattendue	500	`INTERNAL_ERROR`

Corrélation par `request_id`. Un middleware ASGI « pur » lit l'en-tête `X-Request-ID` (s'il est bien formé, 1 à 128 caractères sûrs) ou en génère un, le lie à une `ContextVar` et le renvoie dans la réponse. Un filtre de logging l'injecte dans *chaque* enregistrement, y compris ceux émis par le cœur, qui ignore tout de cette mécanique. Les routes de calcul sont des fonctions synchrones : FastAPI les exécute dans un *thread*, ce qui évite de bloquer la boucle d'événements pendant le bootstrap et propage le contexte au thread (ADR-025).

10.3 Journalisation

Trois niveaux, au format JSON, un événement par ligne, avec des champs stables : les *logs techniques* (accès, cache), les *logs de calcul* (par pilier en niveau `DEBUG`, synthèse du bootstrap, PV par jambe) et le *log d'audit* `calculation_audit`, qui contient les empreintes des entrées, les versions et les options, et permet de rejouer le calcul.

```
{"timestamp": "2026-10-25T00:39:49.930+00:00", "level": "INFO",
 "logger": "core.engines.discounting", "event": "pricing_completed",
 "request_id": "smoke-1", "trade_type": "ois_swap", "currency": "EUR",
 "present_value": 1902.5377349308692, "cashflow_count": 10,
 "par_rate": 0.02254000000000001, "duration_ms": 0.567}
```

Listing 7 – Événement de calcul (champs tronqués).

10.4 SDK et tableau de bord

Le `PricerClient` (basé sur `httpx`) expose `price`, `build_curves`, `catalog`, `list_snapshots`, `get_snapshot`, `health`, `ready` et `version`, et renvoie des objets typés qui réutilisent les schémas de l'API. Chaque code d'erreur devient une exception typée portant le `request_id`, à communiquer au support ; une API injoignable lève `ApiUnavailableError`.

```
from rate_derivatives_pricer.sdk import PricerClient, ois_swap

with PricerClient("http://localhost:8000") as client:
    priced = client.price("EUR_2026-10-19", ois_swap(10_000_000, 0.0225, "pay_fixed", "5Y"))
    print(priced.result.present_value, priced.result.par_rate, priced.request_id)
```

Listing 8 – Utilisation du SDK.

Le tableau de bord Streamlit offre deux onglets : la courbe (zéros, forwards, facteurs d'actualisation, table de calibration) et un formulaire de pricing d'OIS (PV, taux de par, annuité, PVBP, flux). Il *ne calcule rien* : tout passe par le SDK. Le SDK sert ainsi de test d'intégration vivant, puisqu'une rupture du contrat HTTP casse immédiatement le tableau de bord. Le contrat `import-linter` (3) interdit à `ui` d'importer `api` directement ; le SDK réexporte les schémas de l'API (`sdk/models.py`), ce qui rend l'import indirect `ui` → `sdk` → `api.schemas` légitime (ADR-021).

11 Validation et qualité

11.1 QuantLib comme oracle

QuantLib (version 1.44) n'est utilisée que comme *oracle de test* : dépendance de développement uniquement, jamais importée par le cœur (ADR-003). Le module `quantlib_oracle.py` traduit nos conventions en objets QuantLib (`DepositRateHelper`, `OISRateHelper`, `PiecewiseLogLinearDiscount`, `MakeOIS`). Comme QuantLib possède une date d'évaluation *globale*, un gestionnaire de contexte la positionne sur la date du snapshot puis restaure la valeur précédente.

Les cas sont *générés à partir de grilles*, non écrits à la main :

— **Marchés** : 3 dates de valorisation (dont une fin de mois et la veille de Pâques) × 3 chocs parallèles (−250, 0, +150 points de base), soit 9 marchés ;

— **Swaps spot** : 13 maturités (1W à 30Y) × 2 sens × 2 notionnels × 3 taux fixes × 9 marchés, soit 1 404 swaps, plus des swaps déjà commencés et à départ différé ;

— **Calendrier** : jours ouvrés TARGET de 1995 à 2060 comparés jour par jour ; échéanciers : fréquences × règles × fin de mois × deux ans de dates de départ.

11.2 Tolérances et résultats

Les tolérances sont des constantes nommées d'un seul module de test (module `tolerances.py` du dossier de validation) ; les écarts réellement constatés sont consignés dans le journal des décisions (ADR-024).

Table 4 – Écarts maximaux avec QuantLib (9 marchés, 1 404 swaps spot).

Grandeur	Critère	Écart mesuré
Calendriers, décomptes, échéanciers	égalité exacte	exacte
Facteurs d'actualisation, log-linéaire	$< 10^{-10}$	$2{,}5 \times 10^{-13}$
Facteurs d'actualisation, linéaire sur zéro	$< 10^{-10}$	$2{,}3 \times 10^{-11}$
PV rapportée au notionnel	$< 10^{-8}$	$2{,}3 \times 10^{-13}$
Taux de par (relatif)	$< 10^{-8}$	$9{,}9 \times 10^{-11}$
Annuité (relatif)	$< 10^{-8}$	$5{,}6 \times 10^{-14}$
PV au taux de par (propriété)	$\leq 10^{-10} \times N$	respecté

La PV est comparée *relativement au notionnel* : un swap proche du par a une PV proche de zéro, différence de deux jambes de plusieurs millions, pour laquelle un écart relatif à la PV n'a pas de sens. La figure 4 montre l'écart sur les facteurs d'actualisation le long de la courbe de référence, entre les piliers et au-delà du dernier.

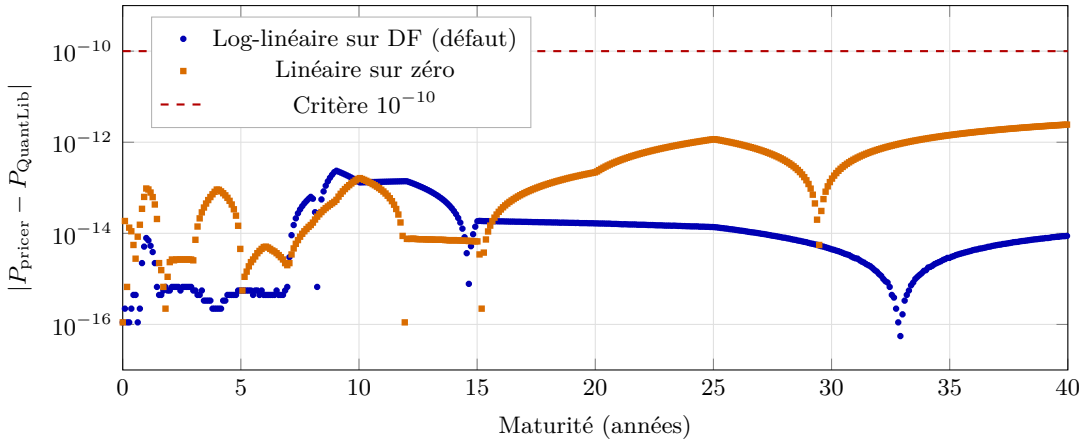


Figure 4 – Écart absolu sur les facteurs d'actualisation, 0 à 40 ans, snapshot de référence. Les écarts nuls sont tracés au plancher de l'échelle (10^{-17}). Maximum : $2{,}4 \times 10^{-13}$ (log-linéaire) et $2{,}4 \times 10^{-12}$ (linéaire sur zéro).

11.3 Pyramide de tests

Niveau	Contenu
Unitaires	Calendriers, décomptes, échéanciers, interpolateurs, solveurs, coupons, registres, cache, mappers ; cas limites et erreurs.
Propriétés (`hypothesis`)	Sur des centaines d'entrées aléatoires : $P(t_0) = 1$, facteurs décroissants, cohérence forward/DF, **PV nulle au taux de par**, symétrie payeur/receveur, résidus de bootstrap sous chocs.
Validation QuantLib	Comparaison systématique sur les grilles ci-dessus (75 tests).
API / SDK / UI	Contrat HTTP, erreurs, `request_id` jusque dans les logs du cœur, cache, SDK typé, tableau de bord exécuté pour de bon (`AppTest`).
Intégration	SDK → HTTP → cœur donne *exactement* les mêmes nombres que le cœur appelé directement ; *golden file* de non-régression.

Au total, **265 tests**, avec une couverture de **99 %** pour `core/` (96 % pour l'ensemble du paquet). Le *golden file* fige la PV, le taux de par et l'annuité de cinq trades d'exemple ainsi que les facteurs d'actualisation de chaque pilier.

11.4 Chaîne de qualité

Avant chaque commit, un hook `pre-commit` exécute `ruff` (style, docstrings Google, interdiction de `print`), `mypy` (mode strict sur `core` et `services`), `import-linter` et les tests unitaires. La CI GitHub Actions rejoue la même boucle sur une machine Linux vierge, puis l'ensemble des tests avec mesure de couverture ; la protection de branche exige qu'elle soit verte avant toute fusion.

11.5 Incidents rencontrés

Quatre incidents, instructifs sur la méthode, sont consignés :

1. **Règle de fin de mois.** La première version du générateur déplaçait aussi la date de départ ; QuantLib ne le fait pas. *Les détails de convention se vérifient date par date contre un oracle, ils ne se devinent pas.*

2. **Extrapolation du schéma linéaire sur le zéro.** Linéaire dans la première version, à forward constant chez QuantLib ; l'écart n'apparaissait qu'au-delà de 30 ans. *Il faut aussi tester hors des piliers.*

3. **Contrat `import-linter` trop large.** Celui de la phase 0 interdisait à `ui` d'importer `api`, même indirectement, ce qui empêchait le SDK de réutiliser les schémas de l'API. Il a été scindé en deux contrats plus précis et la décision est documentée (ADR-021).

4. **Golden file sous Linux.** Sur la CI, la PV du swap receveur à 30 ans (−1 034,69 €, différence de deux jambes d'environ 2,7 millions) s'écartait de 2×10^{-9} € de la valeur calculée sous Windows, à cause des arrondis d'`exp` et `log` propres à chaque plate-forme : $2{,}2 \times 10^{-12}$ en relatif à la PV, au-dessus de la tolérance relative de 10^{-12}. Les PV du golden file sont désormais comparées par unité de notionnel.

12 Journal des décisions

Les décisions non triviales sont consignées sous forme d'ADR (*Architecture Decision Records*) dans `docs/DECISIONS.md`, dans la même PR que le code concerné. Celles de la phase 1 :

ADR	Décision
019	Conventions EUR €STR : TARGET, spot lag 2, flux annuels Act/360, Modified Following, fin de mois, payment lag 1, sans lookback ; dépôt overnight ; deux variantes 30/360.
020	Fixings : obligatoires avant t_0, fixing du jour utilisé s'il est fourni, télescopage de la partie projetée, flux échus exclus.
021	Frontières entre couches : schémas partagés API/SDK, entrées du service en dataclasses, contrat `ui` scindé.
022	Bootstrap : pilier d'un OIS = dernière date de paiement, passe unique, encadrement $[-50\,\%, +100\,\%]$, Brent par défaut, extrapolation à forward constant.
023	Données de marché synthétiques.
024	Validation QuantLib : tolérances inchangées, écarts mesurés, PV comparée au notionnel.
025	Contrat HTTP : enveloppe, empreintes, identifiant de requête, erreurs inattendues, variables d'environnement, routes synchrones en threads.
026	Clé du cache : SHA-256 des données, de l'interpolation, du solveur et de la version ; nom exclu.

13 Limites et suite

Limites assumées de la phase 1.

— **Courbe unique.** L'actualisation et la projection partagent la courbe €STR. Un swap contre Euribor exige une courbe de projection distincte [2, 1] (phase 2).

— **Capitalisation simple.** Ni *lookback*, ni *lockout*, ni *observation shift*.

— **Décalage de paiement sans ajustement de convexité** : le coupon est évalué sur la courbe puis actualisé à sa date de paiement ; l'écart est cohérent avec le moteur d'actualisation de QuantLib, sur lequel la validation s'appuie.

— **Une seule devise** (EUR) ; l'USD (SOFR, calendrier US) s'ajoutera par configuration.

— **Données synthétiques** : le snapshot peut être remplacé par des cotations réelles sans changement de code.

— **Pas de risque** : seuls le PVBP et l'annuité sont fournis.

Prochaines étapes. La phase 2 introduit l'`IborIndex` (Euribor 3M/6M), le bootstrap multi-courbes (actualisation OIS, projection Euribor), le swap vanille (IRS) et le risque par *bump-and-reprice* sur les instruments de calibration : DV01 parallèle, DV01 par bucket (*key-rate*), convexité et DV01 par courbe. L'architecture a été conçue pour cela : le risque dépendra d'une fonction de valorisation abstraite, de sorte qu'ajouter une mesure ne demandera aucune modification des moteurs.

A Calibration détaillée de la courbe

Table 5 – Piliers calibrés sur le snapshot synthétique du 2026-10-19 : cotation, facteur d'actualisation, taux zéro continu, itérations de Brent et résidu final (taux décimal).

Pilier	Type	Date	Cotation (%)	DF	Zéro (%)	Itér.	Résidu
ON	Dépôt	2026-10-20	1,921	0,99994664	1,948	5	$-6{,}0 \times 10^{-15}$
1W	OIS	2026-10-29	1,922	0,99946637	1,948	6	$-8{,}4 \times 10^{-16}$
2W	OIS	2026-11-05	1,921	0,99909360	1,947	6	$-3{,}4 \times 10^{-16}$
1M	OIS	2026-11-24	1,917	0,99808630	1,942	7	$4{,}8 \times 10^{-16}$
2M	OIS	2026-12-22	1,905	0,99662389	1,929	7	$-1{,}3 \times 10^{-15}$
3M	OIS	2027-01-22	1,893	0,99502796	1,915	7	$2{,}9 \times 10^{-16}$
6M	OIS	2027-04-22	1,874	0,99045842	1,892	7	$-7{,}4 \times 10^{-16}$
9M	OIS	2027-07-22	1,871	0,98585436	1,884	7	$4{,}9 \times 10^{-16}$
1Y	OIS	2027-10-22	1,885	0,98109021	1,894	7	$-3{,}0 \times 10^{-16}$
18M	OIS	2028-04-24	1,934	0,97095063	1,946	8	$-5{,}2 \times 10^{-17}$
2Y	OIS	2028-10-24	1,992	0,96046173	2,001	7	$-1{,}4 \times 10^{-16}$
3Y	OIS	2029-10-23	2,098	0,93845492	2,108	8	$-6{,}9 \times 10^{-18}$
4Y	OIS	2030-10-22	2,183	0,91575196	2,194	7	$7{,}6 \times 10^{-17}$
5Y	OIS	2031-10-22	2,254	0,89261592	2,267	7	$1{,}4 \times 10^{-17}$
6Y	OIS	2032-10-22	2,318	0,86908808	2,333	7	$-3{,}5 \times 10^{-18}$
7Y	OIS	2033-10-24	2,379	0,84512772	2,397	7	$-1{,}4 \times 10^{-17}$
8Y	OIS	2034-10-24	2,436	0,82115870	2,457	7	$2{,}1 \times 10^{-17}$
9Y	OIS	2035-10-23	2,488	0,79728635	2,513	7	$-2{,}1 \times 10^{-17}$
10Y	OIS	2036-10-22	2,535	0,77357667	2,563	7	$-6{,}9 \times 10^{-18}$
12Y	OIS	2038-10-22	2,612	0,72756799	2,647	7	$3{,}5 \times 10^{-18}$
15Y	OIS	2041-10-22	2,688	0,66362996	2,730	7	0
20Y	OIS	2046-10-23	2,731	0,57388463	2,773	9	$-1{,}0 \times 10^{-17}$
25Y	OIS	2051-10-24	2,702	0,50544470	2,726	8	$-3{,}5 \times 10^{-18}$
30Y	OIS	2056-10-24	2,651	0,45122111	2,650	8	$-1{,}4 \times 10^{-17}$

B Carte des modules

Chemins relatifs à `src/rate_derivatives_pricer/`.

Module	Rôle
`core/errors.py`	Hiérarchie d'exceptions typées.
`core/registry/`	Registre générique et six registres concrets, chargement paresseux.
`core/conventions/business_day.py`	Conventions de jour ouvré.
`core/conventions/tenor.py`	`Tenor`, `Frequency`, arithmétique de dates.
`core/conventions/calendar.py`	Calendriers TARGET, week-ends, nul, joint ; fin de mois.
`core/conventions/day_count.py`	Act/360, Act/365F, 30/360, 30E/360.
`core/conventions/currency.py`	Conventions d'une devise, chargées depuis la configuration.
`core/conventions/data/EUR.json`	

Module	Rôle
`core/schedule/generator.py`	Génération d'échéanciers (accrual, paiement, stub).
`core/market/indices.py`	Description d'un indice overnight.
`core/market/quotes.py`	Cotations avec unité explicite.
`core/market/snapshot.py`	Snapshot immuable : date, cotations, fixings.
`core/market/providers.py`	Fournisseur de fichiers, format canonique.
`core/market/interpolation.py`	Interpolations log-linéaire sur DF et linéaire sur zéro.
`core/market/solvers.py`	Solveurs de Brent et de Newton.
`core/market/curves.py`	Courbe d'actualisation (DF, zéro, forward).
`core/market/environment.py`	Environnement de marché, rapports de calibration.
`core/market/calibration.py`	Instruments de calibration (dépôt, OIS).
`core/market/bootstrap.py`	`CurveBuilder` : bootstrap séquentiel.
`core/instruments/cashflows.py`	Flux polymorphes : coupon fixe, coupon overnight composé.
`core/instruments/legs.py`	Jambe : flux, notionnel, sens.
`core/instruments/swaps.py`	`OisSwap`, `SwapDirection`.
`core/instruments/factories.py`	Dates spot et maturité, construction du swap, factory.
`core/engines/base.py`	Interface de moteur, sélection par registre.
`core/engines/discounting.py`	Moteur d'actualisation d'un swap.
`core/engines/results.py`	`PricingResult` et sous-résultats.
`services/requests.py`	Entrées du service (dataclasses figées).
`services/mappers.py`	Traductions, empreintes, charges utiles JSON.
`services/curve_cache.py`	Cache LRU de courbes, protocole `CurveCache`.
`services/pricer_service.py`	Façade : calculs, métadonnées, diagnostics, audit.
`api/settings.py`	Configuration par variables d'environnement.
`api/schemas.py`	Contrat HTTP (Pydantic).
`api/errors.py`	Traduction des erreurs en réponses structurées.
`api/middleware.py`	`request_id`, journal d'accès, erreurs inattendues.
`api/routes.py`	Routes de la version 1.
`api/main.py`	Fabrique de l'application.
`api/logging_config.py`	Logging JSON et `request_id` (phase 0).
`sdk/client.py`, `sdk/models.py`	Client typé et modèles réexportés.
`ui/views.py`, `ui/app.py`	Présentation (figures, tables) et script Streamlit.

C Reproduire les chiffres et compiler

Le script `docs/latex/generate_data.py` appelle le pricer et écrit, à côté de ce document, les données des figures (`data/*.dat`), les tableaux (`generated/table_*.tex`) et les macros des exemples chiffrés (`generated/numbers.tex`). Pour régénérer puis compiler :

```
uv run python docs/latex/generate_data.py
cd docs/latex
pdflatex pricing_ois.tex
pdflatex pricing_ois.tex      # second passage : table des matieres et references
```

Listing 9 – Régénération des données et compilation.

Références

[1] L. B. G. Andersen et V. V. Piterbarg, *Interest Rate Modeling*, vol. 1, Atlantic Financial Press, 2010.

[2] F. M. Ametrano et M. Bianchetti, *Everything You Always Wanted to Know About Multiple Interest Rate Curve Bootstrapping but Were Afraid to Ask*, SSRN, 2013.

[3] R. P. Brent, *Algorithms for Minimization without Derivatives*, Prentice-Hall, 1973 (chapitre 4).

[4] P. S. Hagan et G. West, *Interpolation Methods for Curve Construction*, Applied Mathematical Finance, 13(2), 2006.

[5] M. Henrard, *Interest Rate Modelling in the Multi-Curve Framework*, Palgrave Macmillan, 2014.

[6] ISDA, *2006 ISDA Definitions*, section 4.16 (conventions de décompte) et section 4.12 (conventions de jour ouvré).

[7] V. V. Piterbarg, *Funding Beyond Discounting : Collateral Agreements and Derivatives Pricing*, Risk, février 2010.